YOLO-MPE: One-stage Macaque Pose Estimation based on Selective State Space

Wanrong Gu[1,2], Yulin Yang[1], Zusheng Zhang[1], Juntao Xiong[1*]

[1]School of Mathematics and Information, South China Agricultural University, Tianhe, Guangzhou, 510642, Guangdong, China.
[2]Southern Marine Science and Engineering Guangdong Laboratory (Zhanjiang), Zhanjiang, 524013, Guangdong, China.

*Corresponding author(s). E-mail(s): 309283022@qq.com;
Contributing authors: guwanrong@scau.edu.cn; yoleen0801@gmail.com; zushengzhang@163.com;

Abstract

Macaque pose estimation is crucial for behavioral analysis in neuroscience and primatology. Existing deep learning-based methods for macaque pose estimation are categorized as top-down and bottom-up. Top-down methods achieve higher accuracy by utilizing spatial constraints from detected bounding boxes, but perform poorly when dealing with occlusions. Bottom-up methods are fast but require complex post-processing steps and have degraded accuracy. To address these issues, we propose YOLO-MPE, a one-stage macaque pose estimation algorithm based on YOLOv8 integrating dynamic multi-scale convolution and State Space Model(SSM).YOLO-MPE directly predicts bounding boxes and associated keypoints end-to-end, reducing post-processing and enhancing occlusion robustness. Our contributions include (1) integrating a keypoint detection branch into the YOLOv8 decoupling header, (2) proposing dynamic multi-scale convolution for better feature extraction, and (3) incorporating the SSM for efficient long-range dependency modeling.Experimental results on the MacaquePose dataset show that YOLO-MPE outperforms state-of-the-art methods, achieving superior accuracy and reduced computational complexity.

Keywords: One-stage Pose Estimation, Macaque Pose Estimation, State Space Models, Dynamic Multi-Scale Convolution

1 Introduction

Macaque pose estimation is crucial for behavioral analysis in neuroscience and primatology [1]. Nowadays deep learning based macaque pose estimation techniques are categorized into two approaches: top-down and bottom-up. Blanco Negrete et al. [2] demonstrated the use of OpenPose for multiple monkey pose estimation, which achieved real-time performance but was less effective, and the bottom-up approach also involves a series of post-processing operations and has a much lower accuracy than the top-down approach. Liu et al. [3] employed HRNet and its variants for macaque pose estimation. Since the input of the top-down method is provided by the detection results of the object detection model, it is generally more accurate due to spatial constraints and can achieve real-time pose estimation when combined with certain real-time object detection models. However, it performs poorly when dealing with occlusion, as the keypoints may be outside the bounding box. YOLO-Pose [4] was proposed as an end-to-end approach, eliminating the post-processing step required by bottom-up methods.YOLO-Pose is a one-stage pose estimation method that directly detects the bounding boxes of multiple objects in the image and their corresponding 2D poses, each of which has a corresponding pose. Furthermore, YOLO-Pose's ability to detect keypoints outside the bounding box can partially mitigate the pose estimation challenge in the occlusion case, in addition to the fact that the YOLO-Pose method is also relatively easy to migrate to existing object detection models with virtually zero increase in computational effort. For example, Dong et al. [5] improved the performance of keypoint detection on YOLOv8 with a well-designed network structure. However, current research on deep learning-based macaque pose estimation focuses on the optimization of CNN and Transformer models, where CNN performs well in local feature extraction but lacks the ability of global context modeling, while Transformer models are capable of global context modeling of features but are limited by quadratic computational complexity. Recently, State Space Models have attracted the attention of a wide range of researchers for their ability to capture long-range dependencies while maintaining linear computational complexity.

In order to overcome the limitations of existing macaque pose estimation techniques, this paper proposes the YOLO-MPE one-stage macaque pose estimation model, which combines dynamic multi-scale convolution and State Space Model on YOLOv8 to achieve efficient macaque detection and pose estimation.

The main contributions of this paper can be summarized as follows:

- A one-stage macaque pose estimation model is proposed by adding a branch in the decoupling head of YOLOv8 for keypoint detection.
- Dynamic multi-scale convolution is proposed, which uses different sized convolution kernels to extract multi-scale information, and uses channel attention to dynamically generate convolution weights before the convolution operation in order to enhance the dependence of the convolution kernel on the input.
- Combining SSM-based VSS module with dynamic multiscale convolution to learn rich contextual information while having linear computational complexity.

- To verify the effectiveness of the proposed YOLO-MPE, we conducted experiments on the MacaquePose dataset [6], and the experimental results show that our model achieves better results compared to other methods.

2 Related Work

2.1 Macaque Pose Estimation

Rollyn Labuguen et al. [7] used CNN and MSE loss for keypoint detection on single monkey images. OpenMonkeyStudio [8] labeled 190,000 macaque datasets from laboratory scenes using a multi-view data augmentation method and used the CPM method to estimate the pose of the macaque after cropping out the macaque's region from each image. In addition, MonkeyPoseKit [9] utilized HRNet [10] to achieve better performance on the OpenMonkeyStudio dataset. Furthermore, Liu et al. [3] introduced polarized self-attention on top of HRNet and employed asymmetric convolution to lighten the macaque pose estimation model.

For the bottom-up approach, Rollyn Labuguen et al. [11] used the open-source pose estimation toolkit DeepLabCut to estimate a single macaque's pose. Additionally, they applied OpenPose to further investigate multiple macaque pose estimation [2]. However, there is currently no one-stage model for multi-macaque pose estimation, highlighting a significant gap in this field. This area warrants further research and exploration.

2.2 Pose Estimation based on the YOLO Series

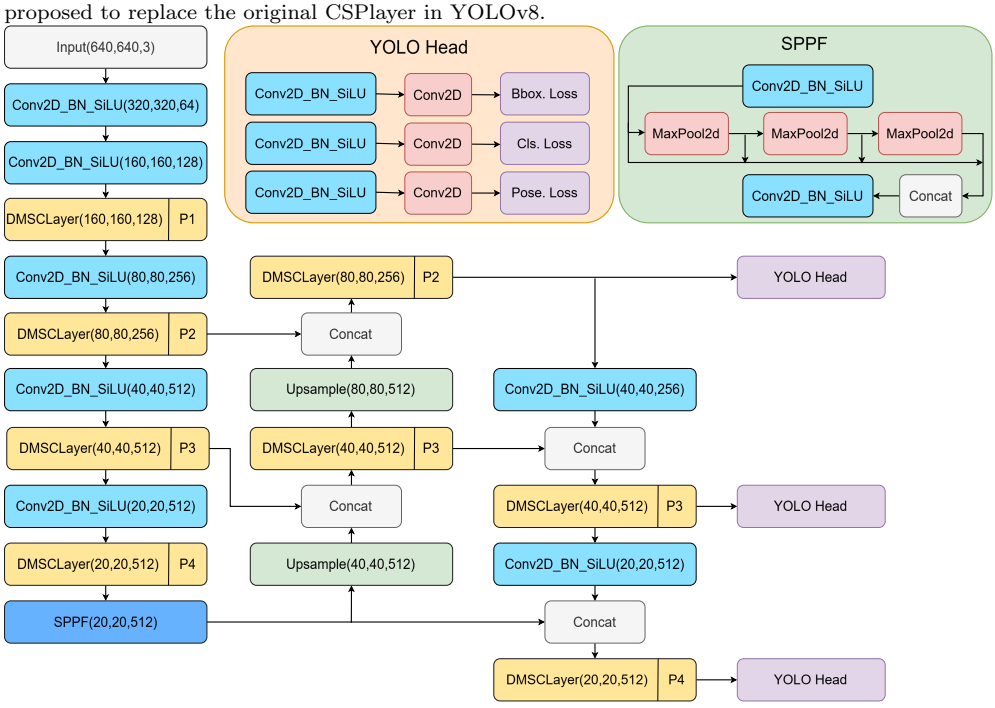
In recent years, deep learning has made significant progress in the field of computer vision. Among them, the YOLO series [12][13][14][15] algorithms have received widespread attention for achieving high accuracy and fast detection speed in target detection.YOLO-Pose is the first model proposed to combine object detection and multi-person pose estimation [4], which achieves simultaneous detection of objects and localization of keypoints by introducing the keypoint similarity loss on the basis of YOLOv5 [12]. To further optimize the performance of pose estimation, researchers have proposed a series of improved YOLO models. For example, SP-YOLO [16] is a lightweight pose estimation network that uses GhostNet instead of convolution in the CSPDarkNet backbone network and introduces a stochastic attention module to enhance the network's perceptual ability. Meanwhile, in order to solve the occlusion problem in pose estimation, other researchers have proposed the method of introducing contextual coordinate attention modules in the backbone and neck networks to enhance the sensitivity of the model to features.YOLO-Rlepose [17] added a window self-attention branch to the C3 module to enhance the model's ability to capture global information. [18] proposed the integration of the SimAM mechanism and the ConvNext structure in YOLOv7 [14] to improve the detection of keypoints under thermal imaging pictures. Similarly, Zheng et al. [19] integrated coordinate attention to YOLOv7 to improve the performance of the keypoint similarity loss function. In summary, most researchers have focused on enhancing the YOLO model for pose estimation tasks using network structures that optimize CNNs or Transformers, and in

this paper, we innovatively combine YOLO with SSM to explore the potential of the model for pose estimation in rhesus monkeys.

2.3 State Space Model on Vision Recognition

The Mamba model [20] was initially proposed in the field of natural language processing and later applied to the vision domain. Vision Mamba [21] follows the Vision Transformer [22] design by replacing attention blocks with Mamba blocks, incorporating positional embeddings in the processing of inputs, and using a bidirectional SSM for extracting visual representations. VMamba [23] introduces a cross-scanning module to address the orientation sensitivity of the SSM. LocalMamba [24] is built on Vision Mamba and introduces a local scanning approach to combine detailed local information within a local window with global context. RSMamba [25] proposes a multipath scanning approach to efficiently integrate the global perceptual field and implement linear complexity modeling. RES-VMAMBA [26] combines Mamba and ResNet [27] in order to utilize both global and local state features, which performs excellently in segmentation tasks. VMRNN [28] leverages the excellent long sequence modeling and spatio-temporal representation capabilities of LSTM [29] and Mamba to optimize the efficiency and accuracy of spatio-temporal prediction. Swin- UMamba [30] combined Swin Transformer [31] with Mamba for pre-training, further contributing to model accuracy for biomedical image segmentation tasks. In this paper, we use

Fig. 1 The network architecture of YOLO-MPE.YOLO-MPE introduces DMSCLayer, which we proposed to replace the original CSPlayer in YOLOv8.



omni-directional scanning based on the VSSBlock proposed by VMamba to enhance the robustness of the model, in addition, we combine the VSSBlock with CNN to enhance local information exchange.

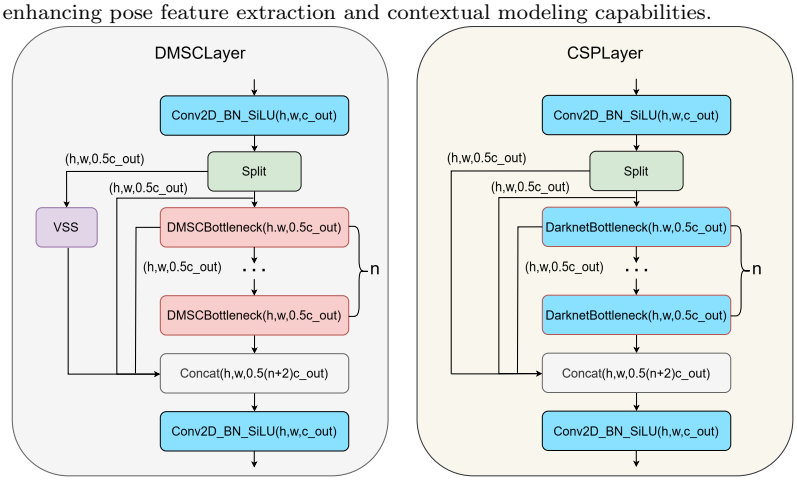
3 Methods

3.1 Overall Architecture

The overall architecture of the YOLO-MPE network is shown in Figure 1. In this paper, we propose a one-stage macaque pose estimation method that incorporates multi-scale dynamic convolution and visual state space(VSS) block using YOLOv8s as the base model. The specific improvements are as follows: (1) Replace DarkNetBottleneck by constructing DMSCBottleneck on YOLOv8 to enhance the ability of capturing the spatial dependence between keypoints. (2) Introduce the VSSBlock in DMSCLayer to realize the complementary fusion of local and global information, and introduce the omni-directional scanning mechanism to enhance the feature-awareness of the model. (3) Introduce scale-invariant Object Keypoint Similarity (OKS) loss in the decoupling head as the keypoint regression, and set specific scale factors for the 17 keypoints of macaque monkey, and use the Binary Cross-Entropy (BCE) loss to compute the confidence of the keypoints.

3.2 Dynamic Multi-Scale Convolution Layer

Fig. 2 Comparison between DMSCLayer and CSPLayer in YOLOv8. DMSCLayer utilizes convolutional kernels of different sizes and incorporates an omnidirectional visual state space module, enhancing pose feature extraction and contextual modeling capabilities.

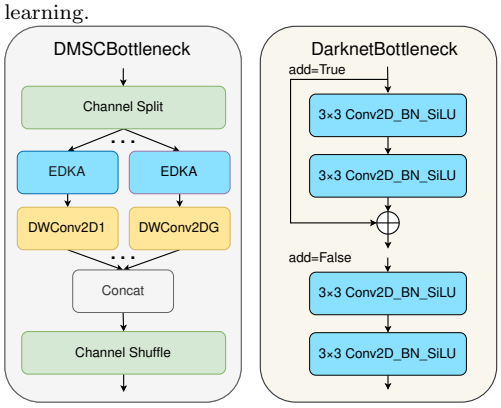


YOLOv8 is widely used for its concise and efficient model structure design, while in the pose estimation task, LitePose [32] and Dite-HRNet [33] demonstrated that larger kernel convolution can achieve better performance. Therefore, we propose DMSCBottleneck, which utilizes convolutional kernels of different sizes to extract richer pose

features. Additionally, we introduce an omni-directional visual state space module to further enhance the model's contextual modeling capabilities. Figure 2 shows the comparison between DMSCLayer and CSPLayer in YOLOv8.

3.3 Dynamic Multi-Scale Convolution Bottleneck

Fig. 3 The computational process in DMSCBottleneck compared to the DarknetBottleneck. EDKA utilizes channel splitting and depth-wise convolution with different sized kernels, enhanced by channel attention to adjust weights dynamically, improving input adaptation and contextual information learning.



While introducing larger convolutional kernels can lead to a greater receptive field, it can also lead to a higher computational cost. Therefore, we use channel splitting to divide the channel into multiple groups, and apply depth-wise convolution in parallel within each group with different sized kernels. It is worth noting that before performing the depth-wise convolution operation, we utilize the channel attention to compute the attention weights of the different groups, and the new weights derived from the element-by-element multiplication of the attention weights with the corresponding weights of the convolution kernels are used for convolutional computation. We call this operation Enhanced Dynamic Kernel Aggregation (EDKA), which enables the model to dynamically adjust the weights of different convolution kernels according to the input data, thus enhancing the input adaptation and dependency of the convolution kernels, and in addition, the introduction of the channel attention enables each convolution kernel to learn rich contextual information.The computational process of EDKA is as follows:

$$\begin{cases} a\left(X\right) = Sigmoid\left(FC\left(Hardsigmoid\left(FC\left(GAP\left(X\right)\right)\right) \cdot X\right)\right) \\ Y = DWConv2d\left(X, a\left(X\right) \cdot w\right) \end{cases} \tag{1}$$

where GAP represents the global average pooling, FC represents the full-connected layer, $Hardsigmoid$ and $Sigmoid$ are the non-linear activation, $DWConv2d$ represents the depth-wise convolution, w is the weight of $DWConv2d$ and is dynamically

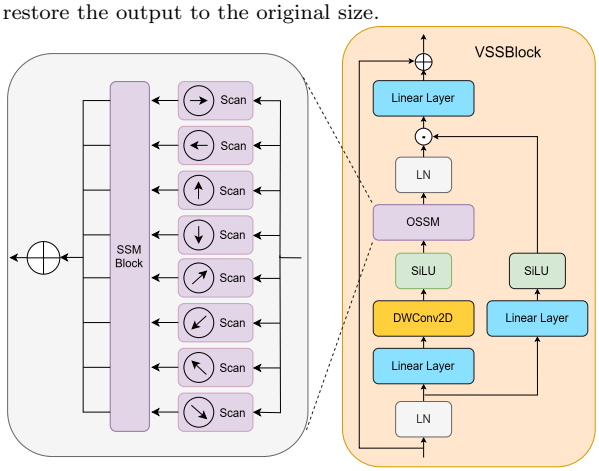
adjusted by $a(X) \in \mathbb{R}^{1\times C}$, and C is a hyper-parameter of $a(X)$ to control the model's performance and learning ability.

$$\begin{cases} [X_1, X_2, ...X_G] = ChannelSplit(X) \\ [Y_1, Y_2, ...Y_G] = EDKA([X_1, X_2, ...X_G]) \\ Y = ChannelShuffle(Cat([Y_1, Y_2, ...Y_G])) \end{cases} \tag{2}$$

In Equation 2, we first split the input tensor X into G groups of channels, resulting in $[X_1, X_2, ..., X_G]$. Then, we apply the EDKA operation to these groups, producing outputs $[Y_1, Y_2, ..., Y_G]$. Finally, we concatenate these outputs and perform a channel shuffle to obtain the final output Y.

3.4 Visual State Space Block

Fig. 4 Structure of the VSSBlock. The OSSM module captures features across eight directions and restore the output to the original size.



Dynamic multi-scale convolution performs feature extraction through convolution kernels of different sizes, which can theoretically capture more features of different scales. However, this approach may not be as effective as fixed 3 × 3 convolution on models with low computational complexity (n and s). To improve the performance of YOLO-MPE under small models and further enhance the feature-awareness of the model, we constructed a visual state space branch in DMSCBottleneck, which efficiently integrates the global contextual information through the complementary fusion of the local and global information, and enhance the feature perceptibility of the model.

3.4.1 Selective State Space Model

State Space Model (SSM) is often used in control systems as a mathematical model to establish the relationship from input to intermediate state to output.The Selective State Space Model originates from Mamba [20], introducing skip connections that

enable the SSM to distinguish and retain relevant information while filtering out irrelevant information. Assuming the system input is $u(t) \in \mathbb{R}$, the intermediate state is $x(t) \in \mathbb{R}^N$, and the output is $y(t) \in \mathbb{R}$ The Selective State Space Model equations are as follows:

$$\begin{cases} x'(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \tag{3}$$

where A $\in \mathbb{R}^{N\times N}$ represents the state matrix, B $\in \mathbb{R}^{N\times 1}$,C $\in \mathbb{R}^{1\times N}$ are projection parameters, and D $\in \mathbb{R}$.

In order to make SSM applicable to deep learning scenarios, SSM is discretized using Zero-Order Hold (ZOH), which is defined as follows:

$$\begin{cases} \overline{A} = \exp(\triangle A) \\ \overline{B} = (\triangle A)^{-1}(\exp(\triangle A) - I) \cdot \triangle B \end{cases} \tag{4}$$

where $\triangle$ is a learnable parameter called the step size. Subsequently we can obtain the new equations Equation 5:

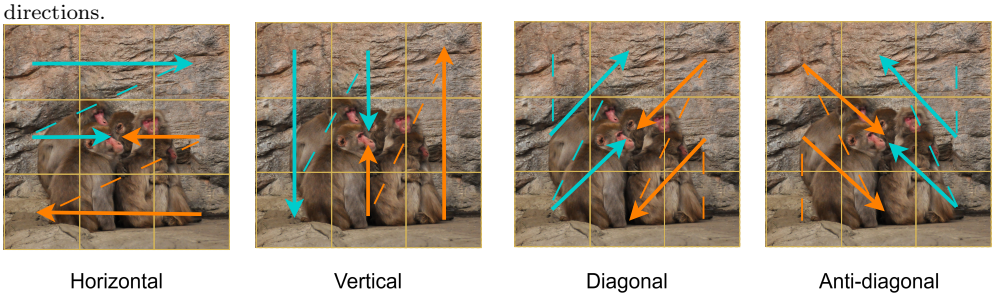
$$\begin{cases} x_k = \overline{A}x_{k-1} + \overline{B}u_k \\ y_k = Cx_k + Du_k \end{cases} \tag{5}$$

3.4.2 Omni-directional Visual State Space Module

Because of the non-causal nature of visual data, directly spreading the image in chunks and inputting it to SSM will suffer from orientation sensitivity. This problem is mitigated by different scanning strategies [21][23][25].Vision Mamba [21] performs forward and reverse scanning, VMamba [23] performs forward and reverse scanning along horizontal and vertical directions respectively, and RSMamba [25] introduces random scanning in addition to forward and reverse scanning. Due to the diverse range of activities and movements of macaques, the posture and movements of macaques can be varied in different angles and directions, which leads to a more diverse distribution of spatial features in macaque images, for this reason, we use the omni-directional selective scanning module (OSSM) to perform bi-directional scans in the horizontal, vertical, diagonal, and anti-diagonal directions, respectively, so as to enhance the model's feature extraction in multiple directions ability of the model to extract features in multiple directions.

The structure of the VSSBlock is shown in Figure 3: first, the input data is processed through layer normalization and then it is divided into two branches. The first branch undergoes linear layer and SiLu activation function processing, while the second branch undergoes linear layer, depth-wise convolution and SiLu activation function processing before entering the OSSM for the next step. After the features are normalized, they are merged with the output of the first branch in an element-by-element product. Next, the features are blended through a linear layer and summed with the residual concatenation to generate the output of the VSSBlock.The OSSM module consists of the scan expand operation, the S6 block, and the scan merge operation. The scan expand operation expands the input image into a sequence along eight directions

Fig. 5 Bi-directional scanning strategies in the horizontal, vertical, diagonal, and anti-diagonal directions.



and extracts features through S6 blocks to ensure comprehensive scanning information and capture diverse features. The scan merge operation then sums and merges the sequences in the eight directions to restore the output image to the input size.The S6 block is constructed based on Mamba, which introduces a selection mechanism based on S4, enabling the model to retain relevant information and filter irrelevant information.

3.5 Loss Function

We follow the YOLO-Pose design and use the OKS loss as the loss function for the keypoints and the BCE loss to compute the confidence of the keypoints. Finally each detection box will predict 57 elements, of which 17 keypoints each keypoint predicts 3 elements $\{x, y, conf\}$ and bounding box 6 elements $\{C_x, C_y, W, H, box_{conf}, cls_{conf}\}$, and the total predicted elements P_v is expressed as follows: $P_v = \left\{C_x, C_y, W, H, box_{conf}, cls_{conf}, k_x^1, k_y^1, k_{conf}^1, ..., k_x^n, k_y^n, k_{conf}^n\right\}$

OKS formula is as follows:

$$\text{OKS} = \frac{\sum_i^n exp\left(-\frac{d_i^2}{2s^2k_i^2}\right) v_i}{\sum_i^n v_i} \tag{6}$$

Here, d_i represents the Euclidean distance between the predicted position of the i-th keypoint and its corresponding ground truth location. s denotes the scale of an object, k_i is a decay constant specific to the keypoint weight, and v_i is the visibility flag for the i-th true keypoint, with $v_i > 0$ indicating that the visibility of the keypoint is greater than 0. The value of OKS ranges from 0 to 1, with values closer to 1 indicating higher similarity.

Then,we use the CIOU loss and BCE loss for bounding box and classes supervision,respectively. The total loss function is as follows:

$$\begin{cases} \mathcal{L}_{cls} = BCE\left(cls_{pred}, cls_{gt}\right) \\ \mathcal{L}_{box} = 1 - CIOU\left(bbox_{pred}, bbox_{gt}\right) \\ \mathcal{L}_{kps} = 1 - OKS\left(kps_{pred}, kps_{gt}\right) \\ \mathcal{L}_{kps_conf} = \sum_{i}^{n} BCE\left(v_i, kps_conf_{pred}^{i}\right) \\ \mathcal{L}_{total} = \lambda_{cls}\mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{box} + \lambda_{kps}\mathcal{L}_{kps} + \lambda_{kps_conf}\mathcal{L}_{kps_conf} \end{cases} \tag{7}$$

where $\lambda_{cls} = 0.5, \lambda_{box} = 7.5, \lambda_{kps} = 12.0, \lambda_{kps_conf} = 1.0$ are the hyper-params used to optimize the model for different scales.

4 Experiments

4.1 Dataset

Our model was trained and evaluated on the MacaquePose dataset [6]. It consists of 13083 macaque images containing 16393 macaques exhibiting various behavioral patterns as well as 17 keypoints. We randomly selected 500 images from the population as a test set and used the rest for model training.

4.2 Evaluation Metrics

We use OKS metrics for evaluation. We report average precision and average recall scores for different thresholds and different object sizes: AP, AP50, AP75, APL, AR.

4.3 Training

We use a similar training strategy as YOLOv8 [15]. Various data augmentation techniques were applied, including random scale adjustment (ranging from 0.5 to 1.5), random shifting ([-10,10]), random flipping with probability 0.5, and mosaic augmentation with probability 1. In addition, we used a SGD optimizer and a cosine scheduler, with the base learning rate set to 1e-2 and a total of 210 epochs of model training.

4.4 Comparisons with state-of-the-art Methods

In this study, we present the YOLO-MPE series of models and compare them with existing state-of-the-art methods on several metrics. Table 1 demonstrates the detailed performance comparison of our models with top-down, bottom-up and one-stage methods. The top-down method benefits from the spatial constraints of the object bounding box and thus can locate the keypoints more accurately, with HRFormer [34] reaching 80.1 AP and TokenPose [35] reaching 95.2 AP on AP50. Since the bottom-up method needs to locate all the keypoints in the input image before grouping them together, it can be found that, compared with the top-down method, the The amount of computation is on the high side while the accuracy of keypoints localization is not high.

Table 1 Comparisons with state-of-the-art methods.

Method	Input	#Params(M)	GFlops	AP	AP50	AP75	APL	AR
Top-down methods								
HRNet[10]	256	28.5	7.7	77.8	92.8	88.2	75.9	80.2
HRFormer[34]	256	43.2	13.7	**80.1**	94.9	**90.9**	**80.3**	**83.8**
TokenPose[35]	256	13.5	5.7	78.7	**95.2**	87.5	77.8	79.1
Bottom-up methods								
AE (HRNet-W32)[36]	640	28.5	64.1	61.3	82.9	66.5	68.3	65.3
HigherHRNet[37]	640	28.6	73.8	**66.2**	**88.0**	**72.3**	**70.2**	65.8
EfficientHRNet[38]	640	23.3	49.6	63.4	83.7	66.8	69.5	**66.3**
One-stage methods								
DEKR[39]	640	29.6	69.6	68.3	88.7	74.9	77.1	72.1
CID[40]	640	29.4	67.5	73.6	91.8	76.7	77.8	74.2
YOLOv5-Pose-s[12]	640	9.6	25.8	58.5	87.4	67.9	59.1	62.2
YOLOv8-Pose-s[15]	640	11.6	30.2	61.0	92.0	70.6	60.9	64.0
YOLOX-Pose-s[41]	640	10.7	18.3	60.7	88.1	66.2	60.1	64.9
YOLO-MPE-n	640	3.3	8.5	55.5	90.8	63.3	57.0	55.8
YOLO-MPE-s	640	11.2	27.0	61.9	93.2	71.6	61.8	63.5
YOLO-MPE-m	640	19.4	54.5	70.1	94.4	86.0	70.3	74.0
YOLO-MPE-l	960	27.0	92.0	**76.0**	**95.7**	**87.7**	**76.4**	**78.7**

Compared with the current state-of-the-art one-stage methods, YOLO-MPE-m has a lead of 1.8 AP over DEKR[39], YOLO-MPE-l has a lead of 2.4 AP over CID[40], and YOLO-MPE-s has an improvement of 3.4 AP, 0.9 AP and 1.2 AP with similar parameter counts compared with the pose estimation models of the YOLO series, such as YOLOv5 [12], YOLOv8 [15], and YOLOX [41], respectively. AP and 1.2 AP. In addition, the YOLO-MPE-series models significantly achieve better results on AP50 than the bottom-up approach.

4.5 Ablation Experiments

4.5.1 Comparison of DarknetBottleneck and DMSCBottleneck

Table 2 Comparisons of DarknetBottleneck and DMSCBottleneck in different scale.

Scale	Method	Input	#Params(M)	GFlops	AP	AP50
n	YOLOv8	640	3.3	9.2	**54.6(+0.7)**	**90.2(+0.5)**
	YOLOv8-DMSC	640	**2.4**	**6.9**	53.9	89.7
s	YOLOv8	640	11.6	30.2	**61.0(+0.3)**	**92.0(+0.2)**
	YOLOv8-DMSC	640	**7.9**	**21.0**	60.7	91.8
m	YOLOv8	640	25.8	80.2	**69.0(+0.1)**	94.5
	YOLOv8-DMSC	640	**17.5**	**55.4**	68.9	**94.6(+0.1)**
l	YOLOv8	640	40.2	127.3	75.3	95.1
	YOLOv8-DMSC	640	**27.8**	**90.4**	**75.5(+0.2)**	**95.1(+0.0)**

In order to validate the greater receptive field and performance enhancement brought by large kernel convolution in the pose estimation task, we replace the DarknetBottleneck module in YOLOv8 with the DMSCBottleneck modules, where in DMSCBottleneck both split the channels into 4 groups, i.e., using 3×3, 5×5,

7×7 and 9×9 convolution kernels for convolution operations. On the Macaque-Pose dataset, the experimental results are shown in Table 2. Although the accuracy of YOLOv8-DMSC decreases by 0.7AP and 0.3AP compared to YOLOv8 under the smaller scale n and s models, respectively, the parameters and computation volume are reduced substantially, and this gap is gradually narrowed and reversed as the model scale increases, e.g., 0.2AP improvement are obtained for YOLOv8-DMSC-l.

4.5.2 Comparison of different scanning strategies

Table 3 Comparison of different scanning strategies.

Scanning Strategies	Input	#Params(M)	GFlops	AP	AP50
Horizontal,Vertical	640	**10.69**	**25.6**	58.5	92.2
Diagonal,Anti-diagonal	640	**10.69**	**25.6**	56.9	90.7
Horizontal,Vertical,Diagonal,Anti-diagonal	640	11.23	27.0	**61.9**	**93.2**

From the experimental results in Table 3, although the diagonal and anti-diagonal scanning strategies are able to capture some additional information, in real scenarios, the macaque's posture may be more along the horizontal and vertical directions, so the diagonal and anti-diagonal scanning strategies are slightly less effective than the horizontal and vertical scanning strategies. The scanning strategy combining horizontal, vertical, diagonal and anti-diagonal can significantly improve the AP and AP50 of the model, which makes the model more robust in dealing with objects with different orientations and poses, and can better adapt to a variety of complex scenes.

4.5.3 Comparison of different resolutions

Table 4 Comparison of different resolutions.

Method	Input	AP	AP50	Inference time(ms)
YOLOv8	640	61.0	92.0	**3.0**
YOLO-MPE	640	**61.9(+0.9)**	**93.2(+1.2)**	6.1
YOLOv8	960	64.2	93.2	**5.2**
YOLO-MPE	960	**65.2(+1.0)**	**93.7(+0.5)**	10.9
YOLOv8	1280	64.7	93.6	**10.0**
YOLO-MPE	1280	**66.0(+1.3)**	**94.3(+0.7)**	15.9

We conduct comparison experiments between YOLOv8 and YOLO-MPE at three resolutions, 640, 960 and 1280, and the experimental results are shown in Table 4. From the experimental data, with the increase of resolution, the detection performance of both YOLOv8 and YOLO-MPE is improved, benefiting from the selective scanning mechanism, YOLO-MPE has a higher accuracy than YOLOv8 under the same resolution, yet leading to an increase in model inference time, it is worth noting that YOLO-MPE can achieve higher accuracy gain with the increase of resolution.

4.5.4 Comparison of different G and C in DMSCBottleneck

Table 5 Comparison of different G and C in DMSCBottleneck, the results of the model with the default configuration we choose are marked with *.

(a)#Params(m)/GFlops					
		Channels			
		1 1 1 1	1 2 2 4	4 4 2 1	4 4 4 4
Groups	1 1 1 1	11.20/27.09	11.22/27.09	11.23/27.09	11.24/27.09
	1 2 2 4	11.20/27.10	11.23/27.10	11.24/27.10	11.27/27.10
	2 2 2 4	11.19/27.12	11.23/27.12	**11.24/27.12***	11.28/27.12
	4 4 4 4	11.21/27.26	11.26/27.26	11.31/27.26	11.36/27.26
(b)AP on the MacaquePose Dataset					
		Channels			
		1 1 1 1	1 2 2 4	4 4 2 1	4 4 4 4
Groups	1 1 1 1	59.8	60.1	60.5	60.8
	1 2 2 4	61.2	61.3	61.6	61.6
	2 2 2 4	61.4	61.5	**61.9***	61.8
	4 4 4 4	61.6	61.8	62.0	61.9

In YOLO-MPE, we can individually set different number of G,C for each DMSC layer to further improve the performance and efficiency of the model. As shown in the figure, we divide the 8 DMSCLayers into four parts, P1,P2,P3 and P4, and the number of channels from P1 to P4 gradually increases while the resolution gradually decreases. A specific G and C are set for each part.The experimental results are shown in Table 5

As the value of G increases, the YOLO-MPE accuracy improves for different values of C with only a small increase in the number of parameters and computation, indicating that a large receptive field is beneficial for the pose estimation task. In particular, for deeper layers (e.g., P4), we chose a larger number of G, while for shallower layers (e.g., P1), a larger number of C was chosen. This design has less impact on the model size and computational complexity, and provides further improvements in model accuracy.

4.6 Visualization of the Detection and Pose Estimation Results

Figure 6 shows the results of pose estimation for macaques using YOLO-MPE. Each image contains one or more macaques, including standing, crawling, and sitting postures, and the model is effective in detection and pose estimation.

In addition, compared with the top-down approach, our method can effectively mitigate the missed detection of keypoints caused by occlusion. In Figure 7, when the keypoints are located outside the detection box due to occlusion, YOLO-MPE is still able to locate the keypoints correctly, which gives it higher generalization ability and better adaptation to various backgrounds and scenes.

Overall, these results show that the YOLO-MPE pose estimation model performs well on the macaque pose estimation task, and is able to accurately locate macaques and estimate their poses in a variety of complex scenarios, providing a reliable basis for further behavioral analysis and research.

Fig. 6 The visualization of macaque pose estimation results.

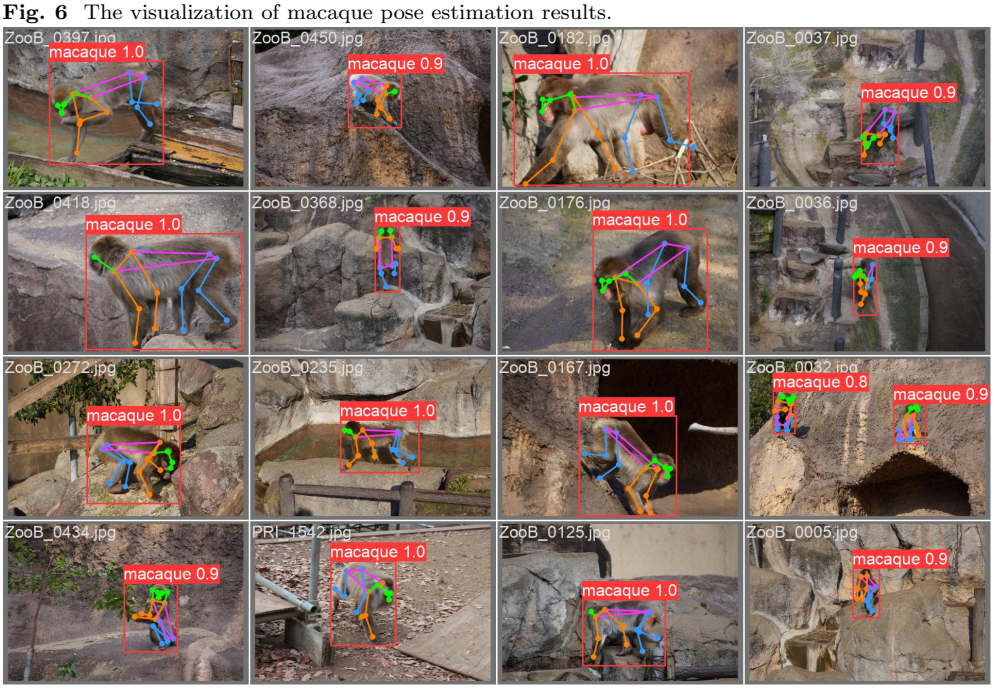
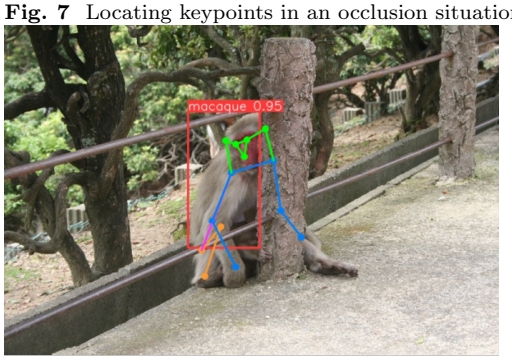


Fig. 7 Locating keypoints in an occlusion situation



5 Conclusion

In this paper, we present YOLO-MPE, a novel one-stage model for macaque pose estimation based on the YOLOv8 model combining dynamic multi-scale convolution and visual state space modules. Our approach addresses the limitations of existing top-down and bottom-up approaches by providing an end-to-end approach that improves the accuracy of macaque pose estimation and robustness to occlusion, while maintaining real-time performance. Extensive experiments on the MacaquePose dataset show that YOLO-MPE outperforms state-of-the-art methods with higher accuracy and

lower computational complexity. Future work will focus on extending this approach to other animal pose estimation tasks and further optimizing the model for application in real-world scenarios of behavioral analysis and wildlife monitoring.

6 Declarations

Competing Interests. The authors have no relevant financial or non-financial interests to disclose.
Authors contribution statement. Yulin Yang: conceptualization, data curation, validation, writing review and editing. Wanrong Gu: conceptualization, funding acquisition, resources, supervision, writing - review and editing. Juntao Xiong: make corrections to the paper. Zusheng Zhang: data curation, validation.
Ethical and informed consent for data used. All datasets used in this paper are public datasets, which can be downloaded through public channels upon request.
Data availability and access. The datasets generated during and/or analysed during the current study are available from the corresponding author on reasonable request.

Acknowledgments

The work was supported by the Special Fund for Marine Economic Development (six marine industries) of Guangdong Province, China (GDNRC[2024]NO.18), the Natural Science Foundation of Guangdong Province (2022A1515011489), the National Statistical Science Research Project (2023LY016), and the National Natural Science Foundation of China (62472182).

References

[1] Hayden, B.Y., Park, H.S., Zimmermann, J.: Automated pose estimation in primates. American journal of primatology **84**(10), 23348 (2022)

[2] Blanco Negrete, S., Labuguen, R., Matsumoto, J., Go, Y., Inoue, K.-i., Shibata, T.: Multiple monkey pose estimation using openpose. bioRxiv, 2021–01 (2021)

[3] Liu, S., Fan, Q., Liu, S., Li, S., Zhao, C.: An attention-refined light-weight high-resolution network for macaque monkey pose estimation. Information **13**(8), 356 (2022)

[4] Maji, D., Nagori, S., Mathew, M., Poddar, D.: Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2637–2646 (2022)

[5] Dong, C., Du, G.: An enhanced real-time human pose estimation method based on modified yolov8 framework. Scientific Reports **14**(1), 8012 (2024)

[6] Labuguen, R., Matsumoto, J., Negrete, S.B., Nishimaru, H., Nishijo, H., Takada, M., Go, Y., Inoue, K.-i., Shibata, T.: Macaquepose: a novel "in the wild" macaque monkey pose dataset for markerless motion capture. Frontiers in behavioral neuroscience **14**, 581154 (2021)

[7] Labuguen, R., Gaurav, V., Blanco, S.N., Matsumoto, J., Inoue, K., Shibata, T.: Monkey features location identification using convolutional neural networks. bioRxiv, 377895 (2018)

[8] Bala, P.C., Eisenreich, B.R., Yoo, S.B.M., Hayden, B.Y., Park, H.S., Zimmermann, J.: Automated markerless pose estimation in freely moving macaques with openmonkeystudio. Nature communications **11**(1), 4560 (2020)

[9] Li, C.-x., Yang, C., Li, Y.-r., Feng, S., Zhang, Z., Wang, Y., Liu, Q., Xiao, X., Xiong, S.: Monkeyposekit: automated markerless 2d pose estimation of monkey. In: 2021 China Automation Congress (CAC), pp. 1280–1284 (2021). IEEE

[10] Sun, K., Xiao, B., Liu, D., Wang, J.: Deep high-resolution representation learning for human pose estimation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5693–5703 (2019)

[11] Labuguen, R., Bardeloza, D.K., Negrete, S.B., Matsumoto, J., Inoue, K., Shibata, T.: Primate markerless pose estimation and movement analysis using deeplabcut. In: 2019 Joint 8th International Conference on Informatics, Electronics & Vision (ICIEV) and 2019 3rd International Conference on Imaging, Vision & Pattern Recognition (icIVPR), pp. 297–300 (2019). IEEE

[12] Jocher, G.: YOLOv5 by Ultralytics. https://doi.org/10.5281/zenodo.3908559 . https://github.com/ultralytics/yolov5

[13] Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., et al.: Yolov6: A single-stage object detection framework for industrial applications. arXiv preprint arXiv:2209.02976 (2022)

[14] Wang, C.-Y., Bochkovskiy, A., Liao, H.-Y.M.: Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7464–7475 (2023)

[15] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLO. https://github.com/ultralytics/ultralytics

[16] Zhang, Y., Wang, Z., Li, M., Gao, P.: Sp-yolo: an end-to-end lightweight network for real-time human pose estimation. Signal, Image and Video Processing **18**(1), 863–876 (2024)

[17] Jiang, Y., Yang, K., Zhu, J., Qin, L.: Yolo-rlepose: Improved yolo based on swin

transformer and rle-oks loss for multi-person pose estimation. Electronics **13**(3), 563 (2024)

[18] Fu, H., Gao, J.: Human fall detection based on posture estimation and infrared thermography. IEEE Sensors Journal (2023)

[19] Zheng, B., Zhang, H., Jin, L.: Research on multi-person pose estimation based on yolo and decoupled multi-level feature layers fusion. In: Proceedings of the 5th ACM International Conference on Multimedia in Asia, pp. 1–7 (2023)

[20] Gu, A., Dao, T.: Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752 (2023)

[21] Zhu, L., Liao, B., Zhang, Q., Wang, X., Liu, W., Wang, X.: Vision mamba: Efficient visual representation learning with bidirectional state space model. arXiv preprint arXiv:2401.09417 (2024)

[22] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929 (2020)

[23] Liu, Y., Tian, Y., Zhao, Y., Yu, H., Xie, L., Wang, Y., Ye, Q., Liu, Y.: VMamba: Visual State Space Model (2024). https://arxiv.org/abs/2401.10166

[24] Huang, T., Pei, X., You, S., Wang, F., Qian, C., Xu, C.: Localmamba: Visual state space model with windowed selective scan. arXiv preprint arXiv:2403.09338 (2024)

[25] Chen, K., Chen, B., Liu, C., Li, W., Zou, Z., Shi, Z.: Rsmamba: Remote sensing image classification with state space model. IEEE Geoscience and Remote Sensing Letters (2024)

[26] Chen, C.-S., Chen, G.-Y., Zhou, D., Jiang, D., Chen, D.-S.: Res-vmamba: Fine-grained food category visual classification using selective state space models with deep residual learning. arXiv preprint arXiv:2402.15761 (2024)

[27] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)

[28] Tang, Y., Dong, P., Tang, Z., Chu, X., Liang, J.: Vmrnn: Integrating vision mamba and lstm for efficient and accurate spatiotemporal forecasting. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5663–5673 (2024)

[29] Graves, A., Graves, A.: Long short-term memory. Supervised sequence labelling

with recurrent neural networks, 37–45 (2012)

[30] Liu, J., Yang, H., Zhou, H.-Y., Xi, Y., Yu, L., Yu, Y., Liang, Y., Shi, G., Zhang, S., Zheng, H., et al.: Swin-umamba: Mamba-based unet with imagenet-based pretraining. arXiv preprint arXiv:2402.03302 (2024)

[31] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 10012–10022 (2021)

[32] Wang, Y., Li, M., Cai, H., Chen, W.-M., Han, S.: Lite pose: Efficient architecture design for 2d human pose estimation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 13126–13136 (2022)

[33] Li, Q., Zhang, Z., Xiao, F., Zhang, F., Bhanu, B.: Dite-hrnet: Dynamic lightweight high-resolution network for human pose estimation. arXiv preprint arXiv:2204.10762 (2022)

[34] Yuan, Y., Fu, R., Huang, L., Lin, W., Zhang, C., Chen, X., Wang, J.: HRFormer: High-Resolution Transformer for Dense Prediction (2021). https://arxiv.org/abs/2110.09408

[35] Li, Y., Zhang, S., Wang, Z., Yang, S., Yang, W., Xia, S.-T., Zhou, E.: Tokenpose: Learning keypoint tokens for human pose estimation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 11313–11322 (2021)

[36] Newell, A., Huang, Z., Deng, J.: Associative embedding: End-to-end learning for joint detection and grouping. Advances in neural information processing systems **30** (2017)

[37] Cheng, B., Xiao, B., Wang, J., Shi, H., Huang, T.S., Zhang, L.: Higherhrnet: Scale-aware representation learning for bottom-up human pose estimation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5386–5395 (2020)

[38] Neff, C., Sheth, A., Furgurson, S., Tabkhi, H.: Efficienthrnet: Efficient scaling for lightweight high-resolution multi-person pose estimation. arXiv preprint arXiv:2007.08090 (2020)

[39] Geng, Z., Sun, K., Xiao, B., Zhang, Z., Wang, J.: Bottom-up human pose estimation via disentangled keypoint regression. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 14676–14686 (2021)

[40] Wang, D., Zhang, S.: Contextual instance decoupling for robust multi-person pose estimation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11060–11068 (2022)

[41] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021 (2021). https://arxiv.org/abs/2107.08430